

COMP 3059 – Capstone Project I**Software Requirements Analysis and Design Assignment****Table of Contents**

1.0 Introduction.....	2
1.1 Purpose.....	2
1.2 Scope.....	2
In-Scope Capabilities.....	2
Out-of-Scope Capabilities.....	3
2.0 System Overview.....	3
2.1 Project Perspective.....	3
2.2 System Context.....	4
2.3 General Constraints.....	4
2.4 Assumptions and Dependencies.....	5
3.0 Functional Requirements.....	6
3.1 Functional Requirement Features.....	6
3.1.1 Maintenance Request.....	6
3.1.2 Meal Planning.....	6
3.1.3 Tracking Personal Expenses.....	7
3.1.4 Bill splitting.....	7
3.1.5 Chore Assignment.....	8
3.2 Use Cases.....	8
3.2.1 Use Case #1 - Submit Maintenance Request.....	8
3.2.2 Use Case #2 - Tracking Personal Expenses.....	9
3.2.3 Use Case #3 - Add a Meal.....	10
3.2.4 Use Case #4 - Bill Splitting.....	10
3.2.5 Use Case #5 - Fix Schedule.....	11
3.2.6 Use Case #6 - User Registration.....	12
3.2.7 Use Case #7 - User Log-in.....	12
3.2.8 Use Case #8 - Profile Manager.....	13
3.2.9 Use Case #9 - Chores Assignment.....	14
3.3 Data Modelling and Analysis.....	15
Normalized Data Model Diagram:.....	15
Activity Diagram:.....	16
Sequence Diagrams:.....	17
UML Class Diagram:.....	19
Data Flow Diagram:.....	19
4.0 Non-Functional Requirements.....	20
5.0 Logical Database Requirements.....	21
6.0 Other Requirements.....	22

1.0 Introduction

This document details the capabilities and constraints of the proposed Dormio system. Developed by Tekwave as a dorm companion app, the system is designed to simplify student life by centralizing key day-to-day tasks such as budgeting, meal planning, and maintenance management. This document defines what the functional and non-functional requirements the system must fulfill, for all project stakeholders, ensuring the delivered solution helps reduce stress and improve organization for students living in residence while supporting staff in managing dorm operations efficiently.

1.1 Purpose

The Software Requirements Specification (SRS) for the Dormio: Dorm Companion App establishes the formal agreement regarding the system's intended capabilities and constraints between the development team (Tekwave) and the project stakeholders. The specification's goal is to accurately define the high-level functional and non-functional requirements that must be met to create the comprehensive dorm companion app.

This document describes the high level software requirements for the system. It describes the what, not how, of the capabilities of the system for the intended audiences. Specifically, the SRS outlines the required functionality across Dormio's core modules: Budget Management (including bill splitting), Customizable Meal Manager, and the Maintenance Requests System.

1.2 Scope

The scope of the Dormio: Dorm Companion App is to deliver a functional Minimum Viable Product (MVP) containing all of the app's core features that directly address key challenges in student life, such as juggling schedules, managing finances, and handling dorm maintenance. The system's primary user groups are Students living in residence and Residence administrators and staff.

In-Scope Capabilities

The following features and deliverables are included within the project's scope:

- **Customizable Meal Manager:** Functionality allowing students to select, save, and modify a personalized meal plan based on preferences.
- **Budget Management System:** A tool for personal finance tracking, including manual transaction entry and bill splitting functionality among assigned roommates.

- **Maintenance Requests System:** A feature allowing students to submit and track maintenance requests, and enabling Residence staff to manage and update the request status.
- **User Management:** Implementation of secure user accounts and profiles for both student and residence staff.
- **Documentation:** Provision of user guides and technical documentation that clearly illustrate application flow, features, and system architecture.
- **Functional MVP:** The final deliverable will be a functional MVP containing these core features.

Out-of-Scope Capabilities

To manage project risks and maintain a focused MVP, the following features are explicitly excluded from this project's scope:

- **Live Payment Processing:** Dormio will not process, integrate with, or initiate any online transactions.
- **Group Communication Features:** Features such as in-app group chat or direct messaging between residents are out of scope.
- **Third-Party Software Integration:** Dormio will not integrate third-party software.
- **Handling of Sensitive Information:** Dormio will not store sensitive data such as financial credentials.
- **Migration of Data:** There will be no automated tools to migrate existing student information into Dormio.

2.0 System Overview

The System Overview section introduces the system context and design.

2.1 Project Perspective

Dormio is a new, self-contained system designed to centralize functions that support a student's daily life. Dormio exists in an ecosystem where apps like Dormefy and Dormbuddy are available. These apps can manage dorm management processes such as meal management and dorm maintenance requests. Dormio differs from these competitors as it handles these essential dorm functions alongside providing students with functions that aid in daily life, consolidating services that are usually spread across multiple applications into one system.

2.2 System Context

Students have a lot of tasks to manage such as handling dorm maintenance, splitting chores amongst residents and managing schedules. Software and applications that address these needs are typically spread across various platforms. Tekwave aims to fill the role of an all-in-one app addressing these needs so that students can handle necessities in a more efficient and less stressful way.

2.3 General Constraints

Business constraints:

Timeline - The milestones and MVP for Dormio spans into early 2026 with the final delivery aimed to be produced by Spring 2026.

Budget – The budget for development is fixed and typically tools required for the project are expected to be free or within budget.

User Adoption and Competition – While Dormio provides many features not found in competitors, Dormio needs to be developed with user adoption and retention in mind

System Constraints:

Internet Connection – Many Features require a reliable internet connection and Tekwave assumes that residents have reliable internet

Security – Sensitive Student information needs to be protected and only provided when necessary.

Platform - Dormio is designed to work with android systems. IOS support will only be added if possible.

Compatibility – Dormio can accept information provided by CVSs to populate non sensitive information

2.4 Assumptions and Dependencies

Assumptions:

Consistent User connectivity – The app assumes a reliable and consistent internet connection for many of its features

User Adoption – residence staff adopt the maintenance system with required access permissions.

Data Security - Security measures are in place to protect sensitive user data with access provided to limited roles and adequate controls are in place.

Positive User Engagement – Dorm Residents will consistently use app features in daily life.

Tech Stack Stability – Tech stack remains accessible throughout development.

Unchanged Project Scope – Requirements will not change after Initial Approval

Time Availability - Team members maintain agreed upon commitments to the project

Dependencies:

Database Connection & Seeding – features/testing blocked until the successful connection and population of test data

Version Control Setup – shared Git repository configured with collaboration being dependant on its setup

Finalized Design Documents – feature implementation depends on sign-off of wireframes and Database schema design

Team Member output – Complex features are dependant on foundational work of other members

Administrative Access Credentials - Staff provides required access

Required External Approval - Staff and management approves of the features and workflow operations.

Resource Allocation – Team commits required hours allocated in the RAM and Gantt Chart.

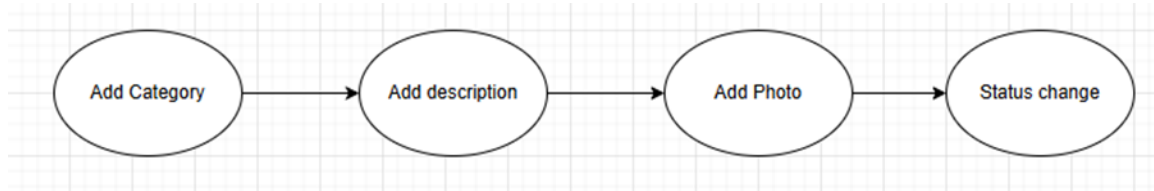
3.0 Functional Requirements

This section describes specific features of the software project. If desired, some requirements may be specified in the use-case format and listed in the Use Cases Section.

3.1 Functional Requirement Features

3.1.1 Maintenance Request

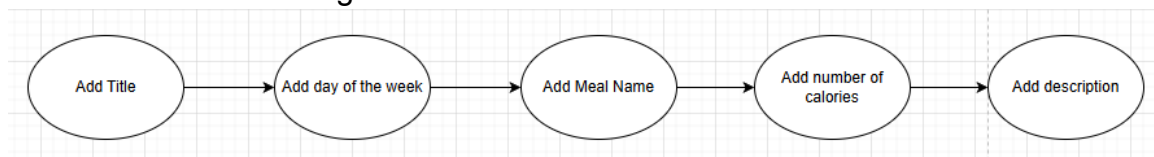
- This feature allows students to request a maintenance for the dorm using a mobile app, and it will automatically send it to the maintenance admin to approve it
- Inputs
 - Issue a category e.g. plumbing, electrical, heater, etc
 - Add a description of the problem
 - Add an optional photo attachment
 - Change status after plumbing
- Processing



- Outputs
 - Completed the form
 - Sent to the maintenance admin for review
 - Update the status of the request

3.1.2 Meal Planning

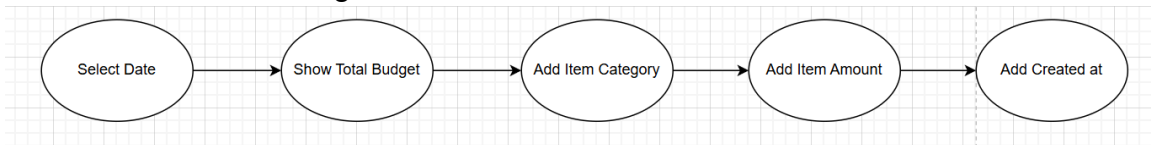
- Allows students to manage their meal onto their schedules. Also, the dining manager can add the name, calories, and description of the meal.
- Inputs
 - Add a title of the meal
 - Add the day of the week it was prepared
 - Add the name of the meal
 - Add the number of calories
 - Add the description of the meal
- Processing



- Outputs
 - Successfully added a meal
 - Students can now add the meal plan to their schedules

3.1.3 Tracking Personal Expenses

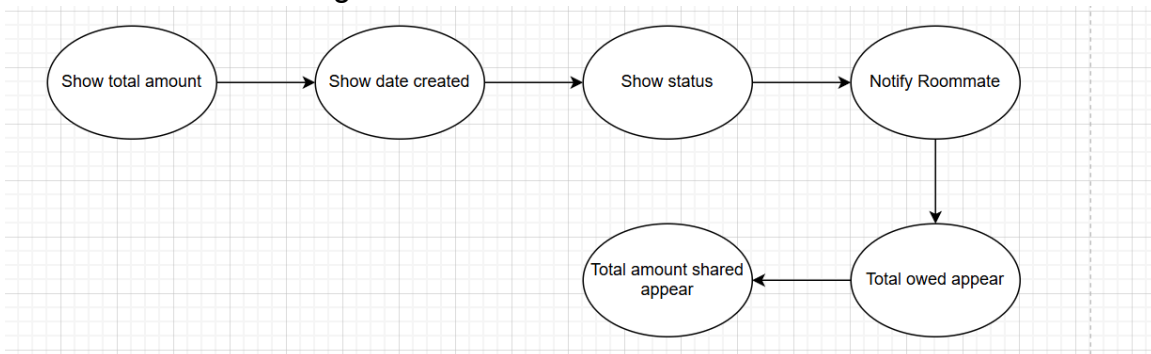
- This feature allows students to track their expenses and manage their budget in a monthly basis
- Inputs
 - Selects the date for the monthly budget
 - Show the total budget
 - Add the category of the item
 - Add the amount of the item
 - Add the date when it was created
- Processing



- Output
 - Total expenses of the student is shown
 - The total budget is shown in the summary page

3.1.4 Bill splitting

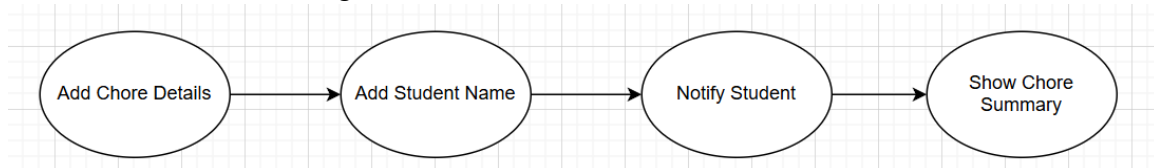
- This allows the student to see and split the bills among roommates in the dormitory.
- Input
 - Total bill amount
 - Select a roommate
 - Add sharing amount
 - Add due date
- Processing



- Output
 - The bill splitting is successful
 - The total amount owed is paid
 - Status changed

3.1.5 Chore Assignment

- Will allow students to assign chores on each roommates
- Input
 - Add name of the task
 - Add description
 - Add status of the chore
 - Specify the student name that will do the chore
- Processing



- Output
 - The students successfully divided the tasks

3.2 Use Cases

3.2.1 Use Case #1 - Submit Maintenance Request

Title: Submit Maintenance Request
Actor: Student and Maintenance Admin
Description: Student submits request for a maintenance issue
Pre-condition: The student must be logged in
Main Success Scenario: 1. Student opens the request page 2. The system displays the forms of the request page 3. Student enters a category of the issue 4. Student inputs the description of the issue 5. Student can add a photo of the issue 6. Student submits the request 7. The system stores the request and generate a ticket 8. The system will show the confirmation page 9. The maintenance admin receives the request 10. The maintenance reviews the request 11. The maintenance crew will change the status to approved
Alternate Flow: A5. The user did not upload the a photo 1. The user submits the request
Error Sequence: E11. The maintenance crew did not approve the request

1. The request will be given a status of denied 2. The sequence may return to the 1st main success scenario
Post Conditions: 1. The request was valid and a maintenance crew will check the request on the specified unit

3.2.2 Use Case #2 - Tracking Personal Expenses

Title: Tracking Personal Expenses
Actor: Student
Description: Student Tracks its expenses
Pre-condition: The student must be logged in
Main Success Scenario: 1. Display the monthly budget of the student 2. Student clicks on the expenses button 3. The student adds a category of expense 4. The student add the amount 5. The student presses the submit 6. The system will autogenerate the date 7. The system will show the summary of total expenses
Alternate Flow: A2. The user presses the billing button 1. The system will redirect the user to the billing page
Error Sequence: E3. The student did not add a category 1. The system will give a validation error to the user 2. The sequence will return to the 3rd main success scenario E4. The student did not add the proper number 1. The system will return a validation error to the user 2. The sequence will return to the 4th main success scenario
Post Conditions: 1. The user was able to add an expenses 2. The system shows the list of expenses with total amount and budget

3.2.3 Use Case #3 - Add a Meal

Title: Add a Meal
Actor: Student, Dining crew
Description: Process of adding a meal to the student's schedule
Pre-condition: The users must be logged in
Main Success Scenario: 1. The dining crew reports the meal in the system 2. The system updates the meal by weekly basis 3. The student selects a meal plan 4. The system updates the student's meal plan
Alternate Flow: A3. The student request an accommodation with food information 1. The dining crew receives the information
Error Sequence: E3. The student request was not accommodated 1. The sequence will continue to the 4th main success scenario
Post Conditions: 1. The student successfully added a meal plan 2. The system successfully updates the database

3.2.4 Use Case #4 - Bill Splitting

Title: Bill Splitting
Actor: Students
Description: Process of splitting the bill
Pre-condition: The users must be logged in
Main Success Scenario: 1. The student goes the billing page 2. The system provides the summary of the bills 3. The student click on the bill splitting 4. The student adds the roommates credential 5. The student adds the divided amount 6. The student adds a due date 7. The student press confirm

8. The roommate receives the bill sharing 9. The roommate accepts the request
Alternate Flow: A4. The student did not add a roommate 1. The sequence will continue to the flow
Error Sequence: E9. The roommate did not accept the amount request 1. The sequence will return to the 3rd main success scenario
Post Conditions: 1. The students successfully split the bills

3.2.5 Use Case #5 - Fix Schedule

Title: Fix Schedule
Actor: Students
Description: Process of a student fixing its schedule
Pre-condition: The users must be logged in
Main Success Scenario: 1. The student presses the schedule 2. The system redirects the student to the schedule page 3. The student checks the class schedule 4. The student adds a work schedule 5. The student adds a study time to the schedule
Alternate Flow: A3. The student checks the meal planning 1. The system will redirect to the flow meal planning if the student did not add it yet
Error Sequence: E4. The student inputs a wrong time for its work and study 1. The system will show an error validation that the time is occupied
Post Conditions: 1. The student successfully fixes its schedule

3.2.6 Use Case #6 - User Registration

Title: User Registration
Actor: Student, Admin Maintenance, Dining crew
Description: Process of users signing-up on the system
Pre-condition: The users must have a valid email address
Main Success Scenario: 1. The users opens the application 2. The users clicks on the register button 3. The users inputs credentials 4. The users clicks submit 5. The system validates the email and password 6. The system sends an email confirmation 7. The users confirms the email 8. The system redirects the user to the accept page 9. The system saves the account to the database
Alternate Flow: A2. The users clicks on the login page 1. The users will be redirected to the login page
Error Sequence: E5. The users credential was validated but was not accepted 1. The sequence will return to the 3rd main success scenario
Post Conditions: 1. The user successfully registered

3.2.7 Use Case #7 - User Log-in

Title: User Log-in
Actor: Students, Admin maintenance, Dining Crew
Description: Process of logging in the system
Pre-condition: The users must be registered on the system
Main Success Scenario: 1. The user opens the application 2. The user is redirected to the login page 3. The user enters the credentials

4. The user press the submit button 5. The system validates the credentials 6. The system redirects the user to its respective page
Alternate Flow: A2. The users clicks on the registration page 1. The users will be redirected to registration page
Error Sequence: E5. The users credential was validated but was not accepted 1. The sequence will return to the 3rd main success scenario
Post Conditions: 1. The user successfully logs in the system

3.2.8 Use Case #8 - Profile Manager

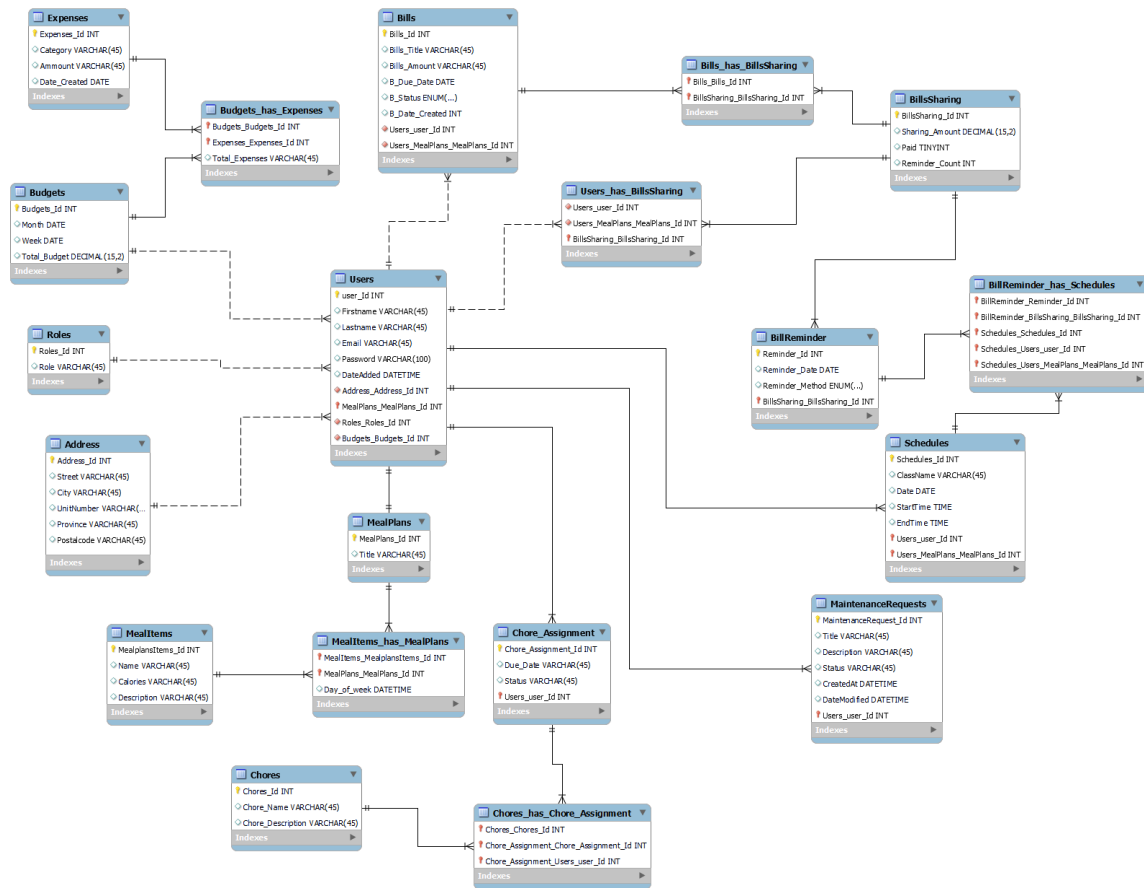
Title: Profile Manager
Actor: Students, Admin maintenance, Dining Crew
Description: Process of editing the profile page of a user
Pre-condition: The users must be logged in
Main Success Scenario: 1. The user goes to the profile page 2. The user edits updates the profile picture 3. The user modifies the email address 4. The user changes the account password 5. The user submits the updated profile information.
Alternate Flow: A3. Invalid Email Format: 1. If the email format is invalid, the system displays an error message and prompts the user to correct it A4. Weak password 1. If the password does not meet the security requirements, the system requests a stronger password
Error Sequence: E5. The updated credential fail validation 1. The sequence will return to the 3rd main success scenario
Post Conditions: 1. The user successfully modifies the profile page

3.2.9 Use Case #9 - Chores Assignment

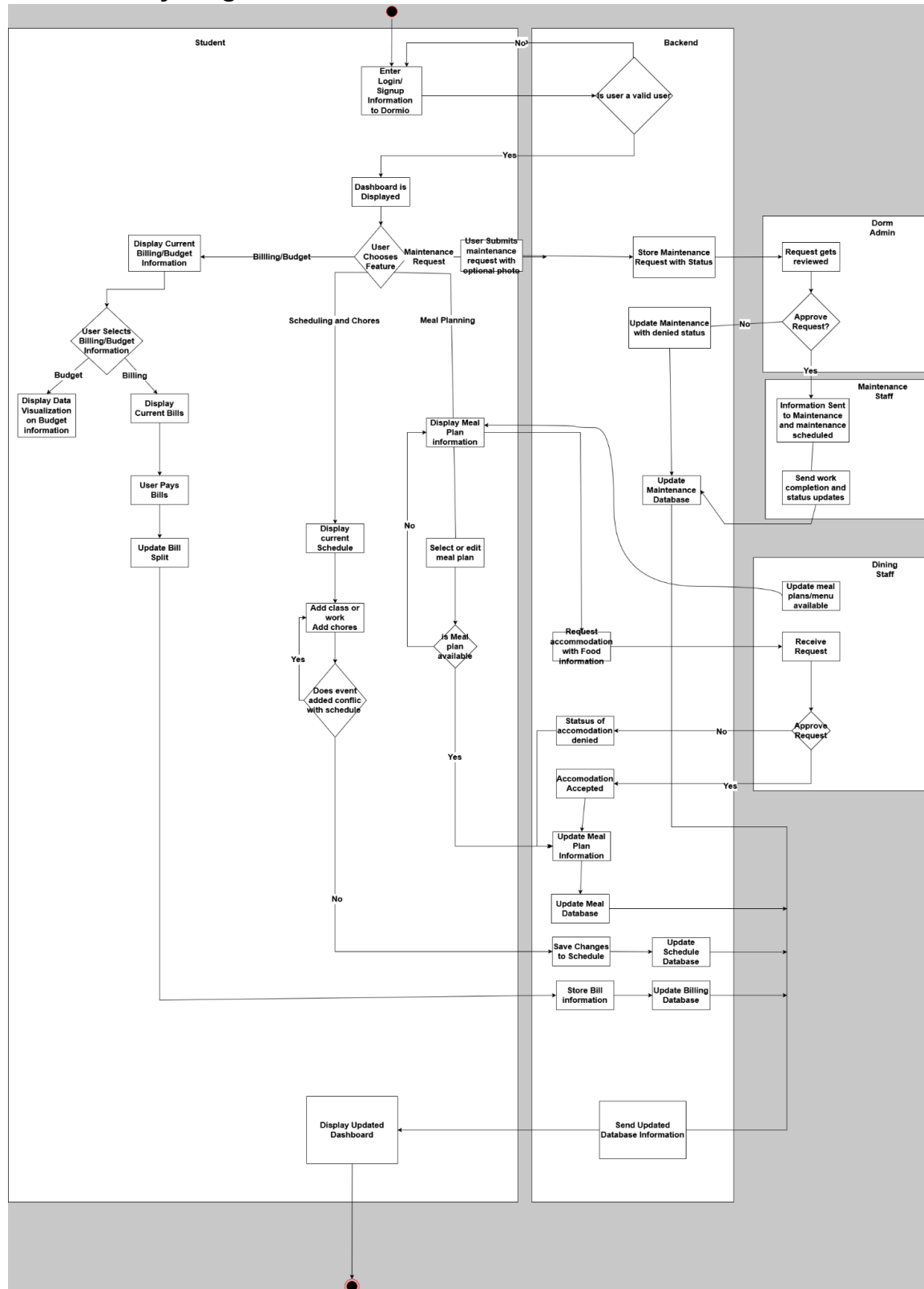
Title: Chores assignment
Actor: Students
Description: Process of dividing chores in the dormitories
Pre-condition: The users must be logged in
Main Success Scenario: 1. The student goes to the chore assignment page 2. The system displays the list of existing chores and options to add more 3. The user selects assign new chores button or plus button 4. The system presents a form for chore details 5. The user enters the chore title 6. The student enters the description 7. The student selects one or more roommates to add in the list 8. The user sets a due date of chore 9. The student confirms the assignment 10. The system saves the assignment to the database
Alternate Flow: A7. The student did not add a roommate: 1. The system will continue its sequence normally
Error Sequence: E10. The system failed to save the chore due to server error 1. The system will display unable to save chore at this time
Post Conditions: 1. The student successfully assign chores to the roommates

3.3 Data Modelling and Analysis

Normalized Data Model Diagram:

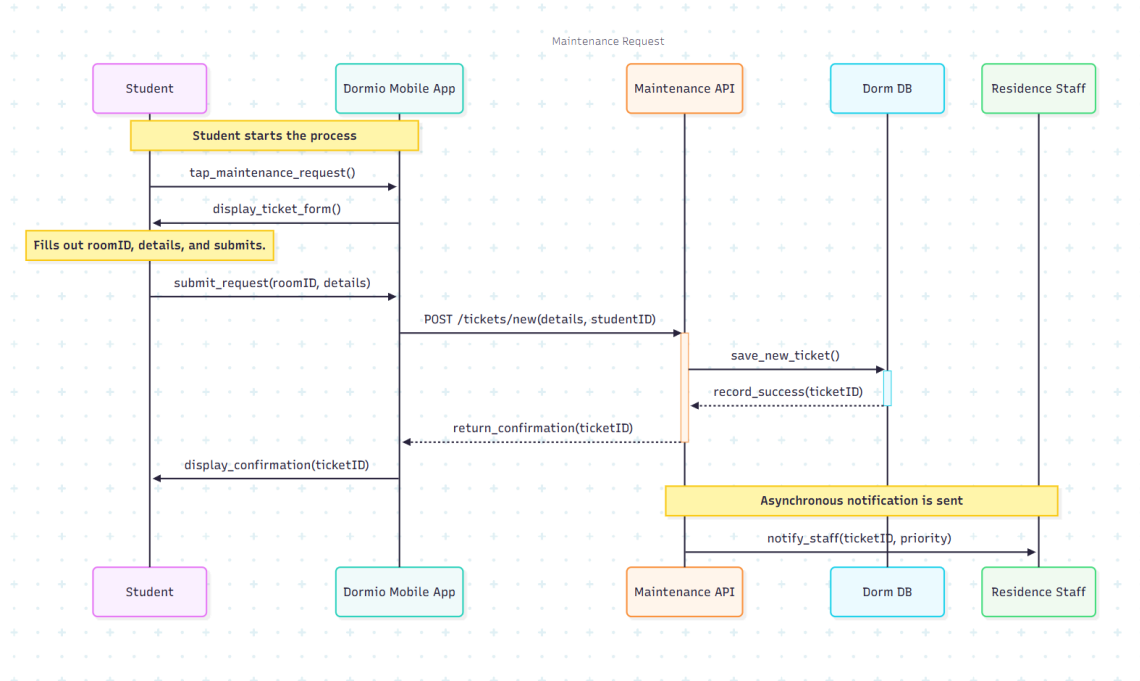


Activity Diagram:

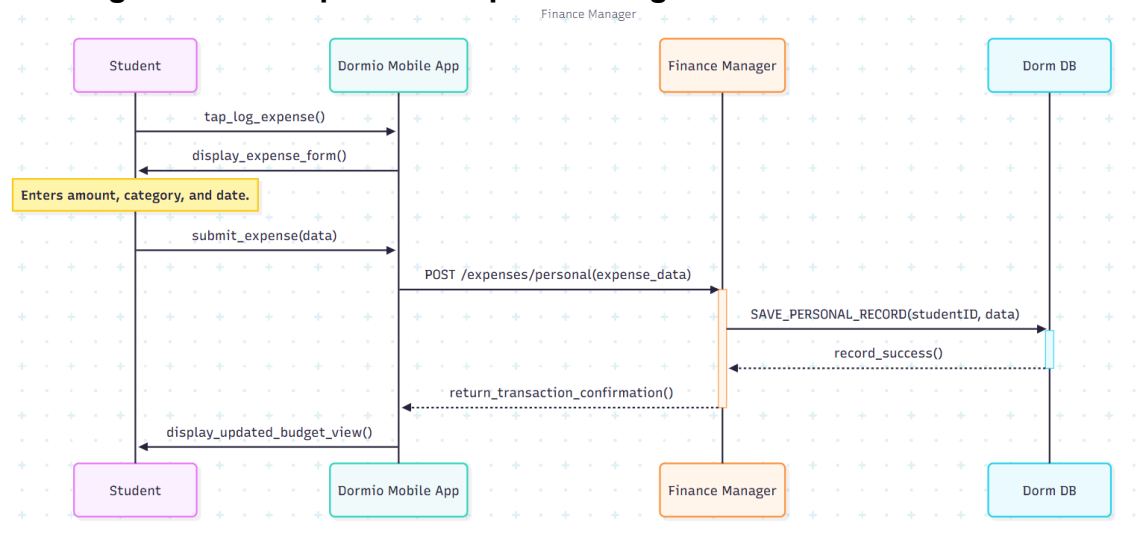


Sequence Diagrams:

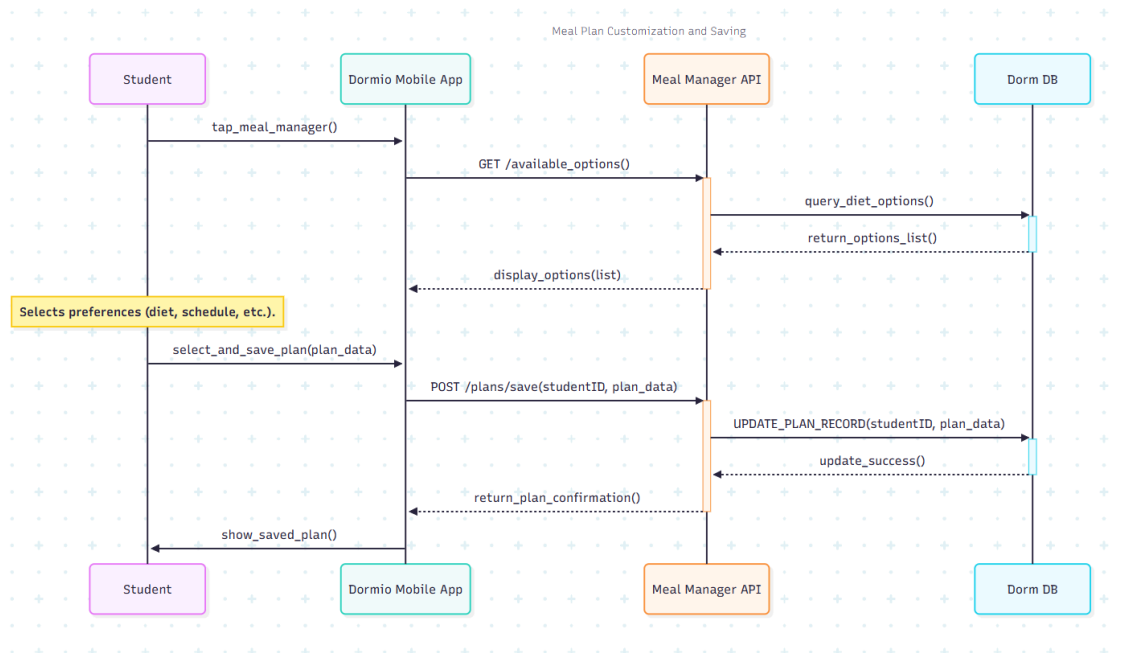
Maintenance Request Sequence Diagram:



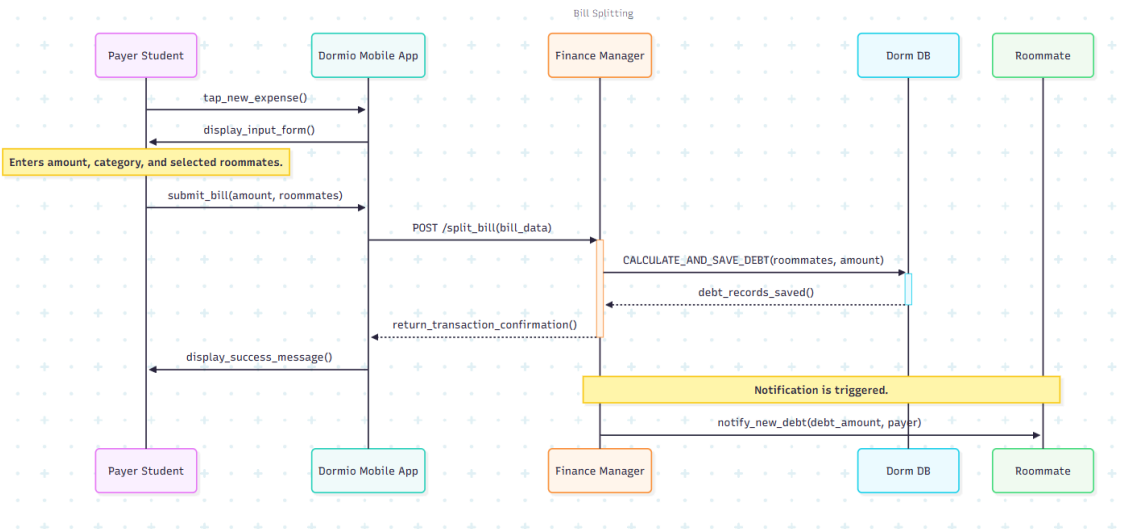
Tracking Personal Expenses Sequence Diagram:



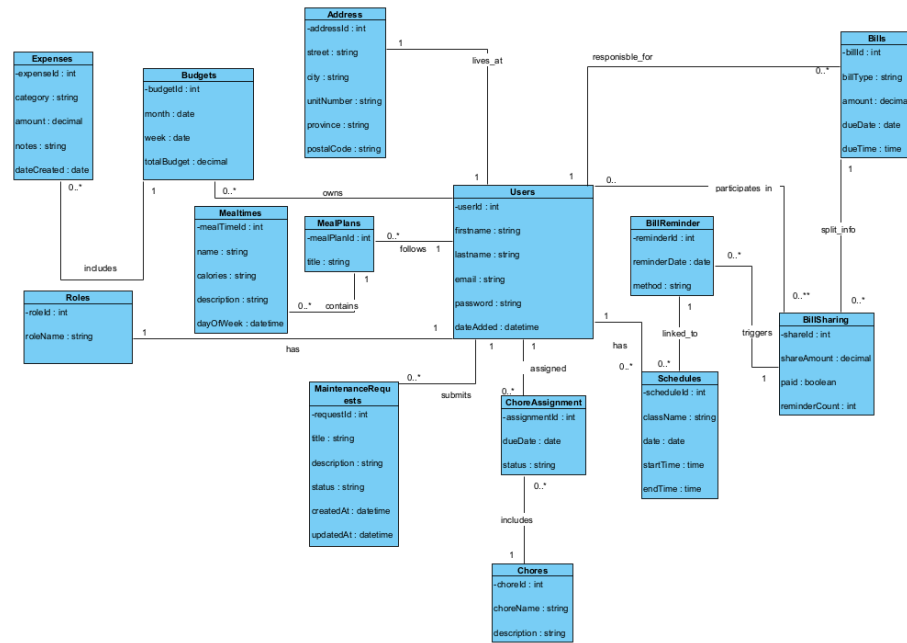
Meal Planning Sequence Diagram:



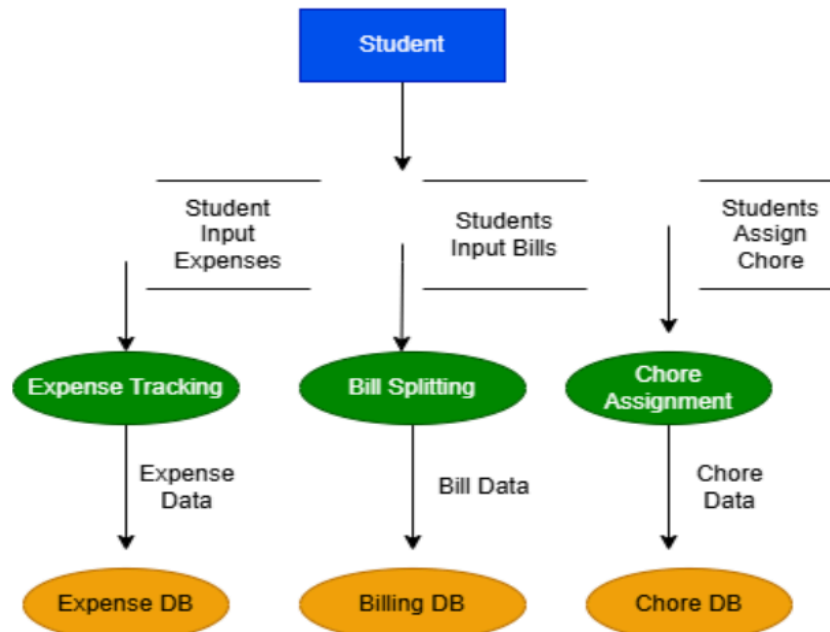
Bill Splitting Sequence Diagram:



UML Class Diagram:



Data Flow Diagram:



4.0 Non-Functional Requirements

- For performance, the app should feel fast. Most actions should load in under one second, and maintenance requests should confirm within two seconds. The system should also support around 150 students using it at the same time without slowing down. Basic database actions like saving and loading data should take no more than half a second.
- For reliability, Dormio should be available 99 percent of the time during the semester. The app should rarely crash, and major errors should happen less than once every 5000 operations. Student data like schedules, meal plans and budgets should always save correctly without losing information.
- For availability, the app should be accessible 24/7. If the system goes down it should be fully restored within five minutes, and no more than ten seconds of recent data should ever be lost.
- For security, students must log in through a secure system and the app should log them out after 15 minutes of inactivity. All data sent between the app and server must be encrypted. Data stored in the system must also be encrypted. Students should only be able to see and edit their own information, while maintenance staff should only access the tickets assigned to them. Any major security updates should be applied within 48 hours.
- For maintainability, the system should be built in a modular way so updates to one feature don't affect large parts of the app. Documentation must be available for all components so future developers can work on it easily. Critical bugs should be fixed within 72 hours, and smaller issues within about a week.
- For portability, the app must run properly on both Android 10+ and iOS 14+ devices. The backend should also be easy to deploy on common cloud platforms with only small configuration changes. Students should be able to export their data in simple formats like CSV or JSON.

5.0 Logical Database Requirements

The Dormio System requires a relation database to store and manage all student, staff and operation data. The following logical requirements define how the system's data must be validated, structured, maintained.

Data Structures Requirements
• The system must use a structured relational model with tables representing each major feature (user, expenses, bills, chores, meals, maintenance).
• Every table must contain a unique primary key and use foreign keys to link related records.
• All date fields must follow a consistent year-month-day format
• Text, numeric and date fields must use appropriate and consistent data types.

Data Integrity Requirements
• Required fields cannot be left empty and numeric fields must contain valid numeric values.
• Input values such as amount, due dates, and statuses must be validated before being stored.

Data Storage Requirements
• The database must permanently store all information related to the user, expenses, bills, chores, meals and maintenance requests.
• Frequently accessed fields must be indexed to ensure efficient search and retrieval.
• The database must be capable of handling continuous data growth throughout the system's usage.

Data Retention Requirements
• Maintenance requests must be stored for audit purposes for a defined retention period.
• Expense, billing and chore records must remain accessible for at least one academic cycle.

Security Requirements
Access to the database must be restricted based on user roles and permissions
All communication between the application and the database must use secure, encrypted connections
Sensitive information such as emails and profiles data must only be viewable by authorized users.

Backup and recovery Requirements
Backup files must be stored securely and protected from unauthorized access
Recovery procedures must allow quick restoration in case of Hardware failure, Security breach or Server crashes

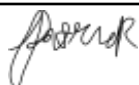
Scalability Requirements
The database must be able to handle an increasing number of users and transactions without performance issues.
The database must support scaling based on system demands.
Query performance must remain stable as datasets expand over time.

6.0 Other Requirements

Additional requirements, if any.

7.0 Approval

The signatures below indicate their approval of the contents of this document.

Project Role	Name	Signature	Date
Developer	Yousuf Salihi		11-16-2025
Developer	Patrick Millares		11-16-2025
Developer	Patrick Solamiilo		11-16-2025
Developer	Ibrahim Abdillahi		11-16-2025
Developer	Bryan Paul Yabut		11-16-2025